

Lab 6

Compute size/offset

BEFORE YOU START:

Be sure your `cVisitor` class has a method for **every** AST node type. If any are missing, you may get difficult-to-debug bad behavior.

Description

For this lab, you will compute the size and offset of all the declarations and variable references. In addition, you will also compute the size of all decls and blocks.

Your implementation must conform to the following requirements:

1. The size and offset must be stored in instance variables with getters/setters (as needed). The getter must simply return the instance variable. The setter must simply set the instance variable. Neither the setter nor the getter are allowed to do any computation.
2. The instance variables must be declared in the lowest member of the class hierarchy where it makes sense to do so. In other words, if all (or most) of the descendants of a class need the feature, it can be declared/implemented in that class. As examples:
 - a. All of the descendants of `cDeclNode` use the size/offset, so the instance variables and getters/setters can be declared there.
 - b. Most of the descendants of `cStmtNode` do NOT need size/offset, so the instance variables and getters/setters must not be visible to `cStmtNode`.
3. You must use the visitor pattern we discussed in class for your implementation.
4. All computation must be done in the visitor subclass. If I do a `diff` of your Lab5 and Lab6 solutions (which I could do through git), the only difference should be the addition of size and offset instance variables to a few classes (with getters and setters), and the addition of a few getters for properties of some classes.
5. You should override `AttributesToString` in each class that has a size and/or offset. `AttributesToString` must return an empty string if both the size and offset are zero.
6. The size for any scope must reflect the high water mark for that scope.
7. You must reclaim and reuse the space for any inner scopes.
8. Anything with a size greater than 1 must be 4-byte aligned.
9. Your implementation must conform to good programming practices. This includes good OOP practices, the use of named constants instead of magic numbers, etc.

`test0` is a concatenation of most of the tests from Lab 4 and Lab 5 with size/offset information added. **Do not start with `test0`.** Instead start with `test1`. When you complete all the other tests, `test0` should pass.

~~Some notes on struct references:~~

- ~~1. The reference $x.y.z$ should have an offset computed as the offset of x plus the offset of y relative to x plus the offset of z relative to y .~~
- ~~2. The reference $x.y.z$ should have a size equal to the size of z .~~

When you start dealing with variable expressions that are arrays, you won't be able to compute offsets. The reason is that, in general, offsets for array references can't be computed until run-time. As a result, we need to handle them differently. The offset for an array reference should be the offset of the variable. The size should be the size of the varref. In addition to computing the size and offset, you also need to compute row sizes so that the final offset computation can be easily generated. Consider the following example:

```
array int[5] a1;
array a1[7] a2;
int v0;
a1 v1;           // size=20 offset=0
a2 v2;           // size=140 offset=20
v1 = v2[1];      // size=20 offset=20 rowsizes=20
v0 = v2[1][2];   // size=4 offset=20 rowsizes=20,4
```

Note: You are only required to handle 1-dimensional arrays in this lab.

Note on correctness criteria for this lab:

There are different valid implementations of `cFuncDeclNode` from last week. These differences will result in differences in this week's correct files. But these differences should NOT affect the size and offset computations. When I grade, if your solution comes up with the same size, offset, and row size values as mine, I will ignore other differences.

Note on completing Lab 5

The assumption at this point is that Lab 5 is complete. This implication is that none of the test files will have syntax nor semantic errors. As a result, if you did not complete Lab 5 in the sense of detecting all specified semantic errors, you do not need to complete it. However, you must be able to generate correct ASTs for valid source files. This includes handling multiple function declarations and correctly handling array references.